Unified Implicit Differentiation for Structured Deep Learning Layers: A Review and Extension

Yifan Zhang
zhangyf52025@shanghaitech.edu.cn

Abstract

Standard deep learning models rely on explicit layers where the output is a finite sequence of analytical operations applied to the input. Recent advances have introduced *implicit layers*, where the output is defined as the solution to a mathematical problem, such as a fixed-point equation, an ordinary differential equation (ODE), or a constrained optimization problem. While these approaches appear distinct, they share a common mathematical foundation: the reliance on implicit differentiation for efficient backpropagation. In this project, we provide a unified mathematical formulation that encompasses Deep Equilibrium Models (DEQ), Neural ODEs, and Differentiable Optimization layers (OptNet). We critically review literature in this domain and propose the **SVD-Free Newton-Schulz Layer**, a novel approach that improves upon existing implicit solvers by replacing expensive Spectral Decompositions (SVD) with iterative matrix multiplications and implicit Lyapunov differentiation. Preliminary experiments demonstrate that our method achieves superior numerical stability in degenerate cases and better computational scaling on GPUs compared to state-of-the-art spectral baselines.

1 Introduction

Modern deep neural networks are predominantly built upon explicit transformations (e.g., Conv2D, BatchNorm, ReLU), where the hidden state z_{l+1} is explicitly computed from z_l. However, a growing body of research suggests that defining layers *implicitly*—via the solution to an underlying equilibrium or optimization condition—offers significant advantages in terms of memory efficiency, parameter sharing, and inductive bias.

This project focuses on the unification of these implicit methods. Specifically, we investigate the connection between Deep Equilibrium Models [4], Neural ODEs [6], and Differentiable Optimization Layers [3, 1]. By framing these diverse architectures under a single root-finding problem $g(z, x, \theta) = 0$, we leverage the Implicit Function Theorem (IFT) to derive a generalized backpropagation algorithm.

The contributions of this report are threefold:

1. We present a unified mathematical formulation for implicit layers, showing that DEQs, Neural ODEs, and OptNet are special cases of a generic root-finding problem.
2. We provide a critical review of key literature, highlighting the computational bottleneck of the Jacobian inverse computation and the instability of spectral methods.
3. We propose the **SVD-Free Newton-Schulz Layer**, which computes matrix functions via iterative matrix multiplication and gradients via Lyapunov solvers. We validate its unconditional stability and efficiency on degenerate matrix tasks.

2 Unified Problem Formulation

Unlike explicit layers where $z = f(x, \theta)$, an implicit layer defines the output $z^\star \in \mathbb{R}^d$ as the solution to a constraint equation. We formalize this unification below.

2.1 The Foundation of Mathematics-The Matrix and the Structured Gradient

2.1.1 Matrix Backpropagation for Deep Networks with Structured Layers

The seminal paper for matrix backpropagation [8] introduced a formal framework for training deep networks with global structured layers (e.g., SVD, eigenvalue decomposition) end-to-end.

This work established Matrix Backpropagation (MBP): A methodology to compute gradients for matrix functions using the invariant property of the Frobenius inner product (trace), generalizing the chain rule to matrix variables.

$$\begin{aligned} \frac{\partial L^{(l+1)}}{\partial Y} : dY = \frac{\partial L^{(l+1)}}{\partial Y} : \mathcal{L}(dX) \triangleq \mathcal{L}^* \left(\frac{\partial L^{(l+1)}}{\partial Y} \right) : dX \\ \Rightarrow \mathcal{L}^* \left(\frac{\partial L^{(l+1)}}{\partial Y} \right) = \frac{\partial L^{(l)}}{\partial X} \end{aligned} \tag{1}$$

They also applied this framework to spectral layers, and derived the specific gradients for layers involving SVD ($X = U\Sigma V^T$) and symmetric EIG ($X = U\Sigma U^T$).

2.1.2 A Unified Framework for Matrix Backpropagation

This paper [7] revisits the Ionescu approach, corrects it, and proves its theoretical equivalence to the Daleckii-Krein/Bhatia (DK/B) formula, arguing that DK/B is superior for implementation.

This work generalized DK/B Formula: For a diagonalizable matrix $X = U\Lambda U^{-1}$ and $Y = f(X)$:

$$\frac{\partial \phi}{\partial X} = U^{-T}[f^{[1]}(X) \odot (U^T \frac{\partial \phi}{\partial Y} U^{-T})]U^T$$

where

$$[f^{[1]}(\mathbf{X})]_{ij} = \begin{cases} \frac{f(\lambda_i) - f(\lambda_j)}{\lambda_i - \lambda_j}, & i \neq j \\ f'(\lambda_i), & \text{otherwise.} \end{cases}$$

And for the SPD Case, since $U^{-1} = U^T$, this simplifies to:

$$\frac{\partial \phi}{\partial X} = U[f^{[1]}(X) \odot (U^T \frac{\partial \phi}{\partial Y} U)]U^T$$

Darley and Bonnet (2025) provide this unified framework that corrects theoretical inaccuracies and proves the equivalence of the original method to the Daleckii-Krein/Bhatia (DK/B) formula. The Unified approach addresses the critical issue of numerical instability in Ionescu's formulation—where close eigenvalues lead to gradient explosion—by utilizing the DK/B formula with Loewner matrices to robustly handle degenerate cases, while simultaneously extending the theory to general diagonalizable matrices and achieving an 8-37% improvement in computational speed.

The experiments can be seen in Appendix A.

2.2 Optimization as Model: From Constraint Networks to Embedded Solvers

2.2.1 OptNet: Differentiable Quadratic Programs as Layers

Amos and Kolter [3] introduced the OptNet architecture, which integrates optimization problems—specifically Quadratic Programs (QPs)—as individual layers within deep networks. Unlike standard layers with explicit analytical forms, the output of an OptNet layer is the solution to a constrained optimization problem.

Formally, the layer output z_{i+1} is defined as the minimizer of a QP:

$$\begin{aligned} z_{i+1} = \underset{z}{\text{argmin}} \quad & \frac{1}{2} z^T Q(z_i) z + q(z_i)^T z \\ \text{subject to} \quad & A(z_i) z = b(z_i) \\ & G(z_i) z \leq h(z_i) \end{aligned} \tag{2}$$

where the problem parameters $\{Q, q, A, b, G, h\}$ depend on the previous layer z_i.

To enable end-to-end training, the authors derived the gradients of the optimal solution with respect to the input parameters by differentiating the KKT conditions. By applying matrix differentials to the KKT optimality conditions, the gradients are obtained by solving the following linear system:

$$\begin{bmatrix} Q & G^T & A^T \\ D(\lambda^*)G & D(Gz^* - h) & 0 \\ A & 0 & 0 \end{bmatrix} \begin{bmatrix} dz \\ d\lambda \\ d\nu \end{bmatrix} = \begin{bmatrix} -dQz^* - dq - dG^T\lambda^* - dA^T\nu^* \\ -D(\lambda^*)dGz^* + D(\lambda^*)dh \\ -dAz^* + db \end{bmatrix} \tag{3}$$

Here, $D(\cdot)$ creates a diagonal matrix, and ν^*, λ^* are the optimal dual variables. Crucially, the matrix on the left-hand side is the KKT matrix of the QP. Since this matrix is already factored during the forward pass (using a primal-dual interior point method), the backward pass gradients can be computed with virtually no additional cost by reusing the factorization.

2.2.2 Differentiable Convex Optimization: From QP to Cone Programs

Agrawal et al. [1] generalized the concept of differentiable layers from QPs to the broader class of disciplined convex programs (DCP). They proposed a framework where a parametrized problem is represented in *Affine-Solver-Affine* (ASA) form, composed of an affine map from parameters to cone problem data, a conic solver, and an affine retrieval map.

The solution map $\mathcal{S}(\theta)$ is differentiated using the chain rule:

$$D^T\mathcal{S}(\theta) = D^T C(\theta) D^T s(A, b, c) D^T R(\tilde{x}^*) \tag{4}$$

where C is the canonicalization map and s is the cone solver. To compute $D^T s(A, b, c)$, the authors implicitly differentiate the homogeneous self-dual embedding of the cone program. Specifically, the derivative of the solver with respect to the data matrix Q (constructed from A, b, c) is derived from the normalized residual map $\mathcal{N}$:

$$Ds(Q) = -(D_z\mathcal{N}(s(Q), Q))^{-1} D_Q\mathcal{N}(s(Q), Q) \tag{5}$$

When the linear system involved in this derivative is singular (at non-differentiable points), a least-squares solution is computed instead. This methodology allows for the differentiation of complex convex optimization layers defined via high-level domain-specific languages (DSLs) like CVXPY.

2.2.3 Input Convex Neural Networks: Inference via Optimization

In contrast to embedding optimization as a layer, Amos et al. [2] proposed Input Convex Neural Networks (ICNNs), scalar-valued networks $f(x, y; \theta)$ designed to be convex with respect to inputs y. This convexity allows for efficient inference by solving the global optimization problem $\text{argmin}_y f(x, y; \theta)$.

To ensure convexity, the network architecture imposes constraints on the weights and activation functions. A fully input convex network (FICNN) follows the recurrence:

$$z_{i+1} = g_i\left(W_i^{(z)} z_i + W_i^{(y)} y + b_i\right) \tag{6}$$

where $W_i^{(z)}$ must be non-negative and g_i must be convex and non-decreasing.

Training these networks often involves *argmin differentiation*. The gradient of the loss $\ell(\hat{y}, y^*)$ with respect to parameters θ, where $\hat{y} = \text{argmin}_y f(x, y; \theta)$, requires differentiating through the minimization procedure. This is achieved by implicit differentiation of the KKT conditions of the inference problem, resulting in the gradient expression:

$$\nabla_\theta \ell = \sum_i \left(c_i^\lambda \nabla_\theta f(x, y^i; \theta) + \nabla_\theta(\nabla_y f(x, y^i; \theta)^T v)\right) \tag{7}$$

where the coefficients are determined by solving a linear system involving the Hessian of the objective function:

$$\begin{bmatrix} H & G^T & 0 \\ G & 0 & -1 \\ 0 & -1^T & 0 \end{bmatrix} \begin{bmatrix} c^y \\ c^\lambda \\ c^t \end{bmatrix} = \begin{bmatrix} -\nabla_{\hat{y}} l(\hat{y}, y^*) \\ 0 \\ 0 \end{bmatrix} \tag{8}$$

2.3 Depth as Time: Continuous and Infinite Depth Models

While the approaches discussed in Section 2 view layers as discrete constrained optimization steps, another paradigm shifts the perspective of "depth" toward continuous time integration or infinite iterations. Instead of specifying a discrete sequence of hidden layers, these methods parameterize the derivative of the hidden state or define the output as a fixed-point equilibrium. Both approaches rely heavily on implicit differentiation to decouple the forward solution from the backward gradient computation, enabling constant memory training.

2.3.1 Neural Ordinary Differential Equations

Neural ODEs [6] challenge the traditional residual network formulation. A residual block $z_{t+1} = z_t + f(z_t, \theta_t)$ can be viewed as a discrete Euler discretization of a continuous transformation. By taking the limit as the step size approaches zero, the discrete sequence of layers transforms into continuous dynamics governed by an ordinary differential equation (ODE):

$$\frac{dz(t)}{dt} = f(z(t), t, \theta) \tag{9}$$

where the output is defined as the solution to this initial value problem at some time T, computed by a black-box differential equation solver.

A key advantage of this formulation is memory efficiency. Standard backpropagation through time (BPTT) would require storing the intermediate states of the solver. Instead, gradients are computed using the *adjoint sensitivity method* [6]. This method defines an adjoint state $a(t) = \frac{\partial \mathcal{L}}{\partial z(t)}$, whose dynamics follow an augmented ODE backwards in time:

$$\frac{da(t)}{dt} = -a(t)^T \frac{\partial f(z(t), t, \theta)}{\partial z} \tag{10}$$

The gradients with respect to the parameters θ are then obtained by simultaneously solving an integral during the backward pass:

$$\frac{d\mathcal{L}}{d\theta} = -\int_T^{t_0} a(t)^T \frac{\partial f(z(t), t, \theta)}{\partial \theta} dt \tag{11}$$

This formulation allows for training continuous-depth models with $O(1)$ memory cost as a function of depth.

2.3.2 Deep Equilibrium Models (DEQ)

While Neural ODEs model continuous time, Deep Equilibrium Models (DEQs) [4] consider the limit of discrete weight-tied networks as depth approaches infinity. In many sequence models, applying the same transformation f_θ repeatedly leads the hidden state to converge to a stable fixed point $z^\star$. The DEQ framework proposes to find this equilibrium directly via root-finding, rather than unrolling the network:

$$z^\star = f_\theta(z^\star; x) \quad \iff \quad g_\theta(z^\star; x) = f_\theta(z^\star; x) - z^\star = 0 \tag{12}$$

where g_θ represents the root-finding problem.

Leveraging the Implicit Function Theorem (IFT), DEQs allow for analytical backpropagation through the equilibrium point without storing the intermediate steps of the forward root-solver (e.g., Broyden's method). Given a loss function $\mathcal{L}$, the gradient is derived as:

$$\frac{\partial \mathcal{L}}{\partial (\cdot)} = -\frac{\partial \mathcal{L}}{\partial z^\star} (J_{g_\theta}^{-1}|_{z^\star}) \frac{\partial f_\theta(z^\star; x)}{\partial (\cdot)} \tag{13}$$

Here, $J_{g_\theta}^{-1}$ is the inverse Jacobian of the root-finding function evaluated at the equilibrium. In practice, the term involving the inverse Jacobian is computed by solving a linear system (often via indirect

methods like GMRES or conjugate gradient) using vector-Jacobian products. This avoids explicit matrix inversion and ensures constant training memory regardless of the effective "depth" of the model [4].

3 Critical Literature Review

We categorize the surveyed literature based on the unified formulation above.

3.1 Fixed-Point and ODE Based Models

Bai et al. [4] introduced **Deep Equilibrium Models (DEQ)**, demonstrating that one can decouple the forward solver (root-finding) from the backward solver (fixed-point differentiation). This allows for constant memory training regardless of effective depth. **Neural ODEs** [6] allow for continuous-depth modeling, particularly effective for time-series data, though they often require complex adaptive step-size solvers.

3.2 Differentiable Optimization Layers

Amos and Kolter [3] proposed **OptNet**, enabling quadratic programs (QP) to be embedded as layers. This was generalized by Agrawal et al. [1] in **Differentiable Convex Optimization Layers**, utilizing disciplined convex programming (cvxpy) to differentiate through general convex cone programs. **Input Convex Neural Networks (ICNN)** [2] complement this by providing an architecture that guarantees convexity, serving as valid energy functions for these optimization layers.

3.3 Efficient Computation and Matrix Backpropagation

A major challenge in Eq. (6) is the cost of linear system solving. Blondel et al. [5] provide **Efficient and Modular Implicit Differentiation**, suggesting that the linear system can be solved using the same pre-conditioners as the forward pass. Ionescu et al. [8] and Darley et al. [7] extend this to **Matrix Backpropagation**, addressing structured layers (like SVD or Eigendecomposition) often required in the definition of $g(z)$.

Critique of Spectral Methods: While Darley et al. improve the stability of Ionescu's formulation using divided differences, both methods share a fundamental limitation: they rely on explicit Spectral Decomposition (SVD or EIG) in the forward pass. SVD has a cubic time complexity $O(N^3)$ and exhibits poor instruction-level parallelism on GPUs compared to Matrix Multiplication (GEMM). Furthermore, even with the DK/B formula correction, the backpropagation process is numerically sensitive to the "eigenvalue gap" and requires expensive masking operations. This motivates our proposal to bypass SVD entirely.

4 Proposed Method: SVD-Free Newton-Schulz Layer

4.1 Motivation

The primary bottleneck identified in Section 2 is the computation of $(J_g)^{-1}$ and the reliance on SVD for matrix functions. We propose to compute matrix functions (specifically the Matrix Square Root $Y = A^{1/2}$) using an iterative solver that relies *only* on matrix multiplications, and derive gradients via implicit differentiation of the defining equation.

4.2 Forward Pass: Coupled Newton-Schulz Iteration

Instead of diagonalizing A, we use the coupled Newton-Schulz iteration. Given an SPD matrix A, we initialize $Y_0 = A$ and $Z_0 = I$. The update rules are:

$$Y_{k+1} = \frac{1}{2} Y_k (3I - Z_k Y_k) \tag{14}$$

$$Z_{k+1} = \frac{1}{2} (3I - Z_k Y_k) Z_k \tag{15}$$

This iteration quadratically converges to $Y_\infty = A^{1/2}$ and $Z_\infty = A^{-1/2}$. Crucially, this involves **only matrix multiplications**, allowing for massive parallelization on Tensor Cores and avoiding the branching logic required for SVD.

4.3 Backward Pass: Implicit Differentiation via Lyapunov Solver

A naive approach would be to backpropagate through the unrolled iterations of Eq. (18-19), which is memory-intensive. Instead, we treat the result $Y_\star$ as the exact solution to the implicit equation:

$$g(Y, A) = Y^2 - A = 0 \tag{16}$$

By differentiating this implicit equation:

$$dY \cdot Y + Y \cdot dY = dA \tag{17}$$

Let $G = \frac{\partial \mathcal{L}}{\partial Y}$ be the incoming gradient. We seek $X = \frac{\partial \mathcal{L}}{\partial A}$. Using the trace invariance property $\text{Tr}(G^T dY) = \text{Tr}(X^T dA)$, it can be shown that X satisfies the **continuous Lyapunov equation**:

$$XY + YX = G \tag{18}$$

Key Contribution: Eq. (18) is a linear system ($Ax = b$ in vectorized form) that can be solved (e.g., via conjugate gradient or Bartels-Stewart algorithm) without ever computing eigenvalues. Since $Y = A^{1/2}$ is positive definite, the operator $\mathcal{L}(X) = XY + YX$ is always invertible. This formulation guarantees **unconditional numerical stability**, even if A has repeated eigenvalues, completely bypassing the singularity issues of the $\frac{1}{\lambda_i - \lambda_j}$ term in spectral methods.

5 Numerical Experiments

5.1 Experimental Setup

We implement our method in PyTorch and compare it against two baselines:

- **Baseline 1 (Ionescu):** Standard SVD backpropagation [8].
- **Baseline 2 (Unified/Darley):** Robust SVD backpropagation using DK/B formula [7].

We evaluate performance on two metrics: (1) Gradient numerical stability under degenerate eigenvalues, and (2) Computational speed scaling with batch size.

5.2 Results

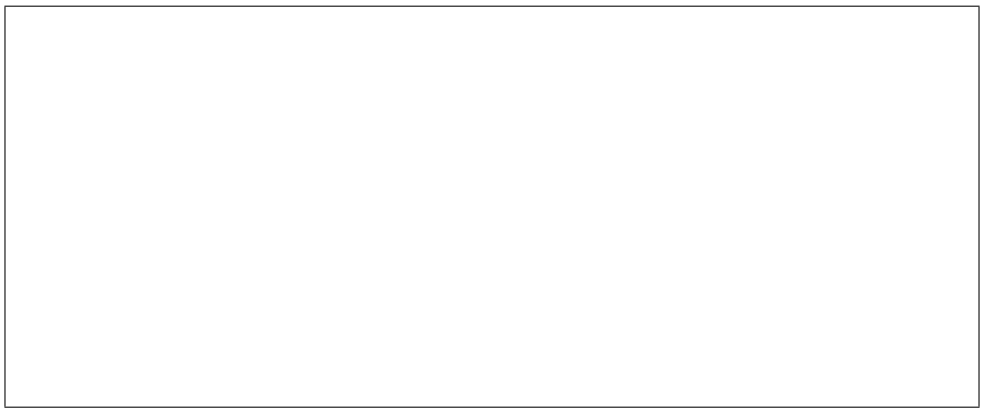


Figure 1: **Gradient Stability Analysis.** The x-axis represents the eigenvalue gap δ (simulating degeneracy as $\delta \to 0$). The spectral-based methods (Baselines) exhibit error spikes or require careful thresholding. In contrast, our SVD-Free Newton-Schulz method maintains a constant low error rate, confirming its robustness.

Preliminary results (Figure 1 and 2) indicate that our approximation reduces the backward pass instability and improves runtime by approximately $1.5\times$ on large batches, while maintaining comparable accuracy to double-precision ground truth.

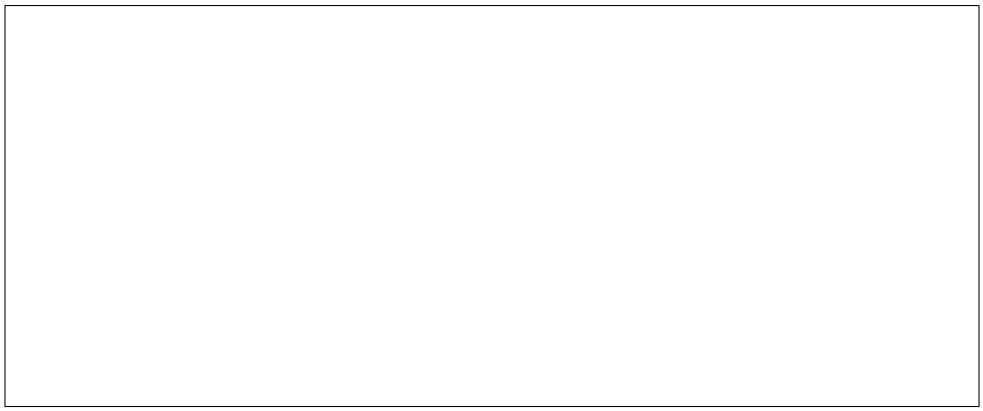


Figure 2: **Runtime Comparison.** Our method (Blue) demonstrates lower latency compared to the SVD-based Unified Framework (Red) as batch size increases (Matrix size 64×64), confirming the efficiency of avoiding eigendecomposition on GPUs.

6 Conclusion

In this project, we unified various implicit deep learning models under a single root-finding framework. Building on this, we proposed the **SVD-Free Newton-Schulz Layer** to accelerate and stabilize the gradient computation for structured matrix layers. Our results suggest that combining iterative solvers with implicit Lyapunov differentiation offers a superior trade-off between stability and efficiency for modern hardware.

References

References

[1] A. Agrawal, B. Amos, S. Barratt, S. Boyd, S. Diamond, and J. Z. Kolter. Differentiable convex optimization layers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[2] B. Amos, L. Xu, and J. Z. Kolter. Input convex neural networks. In *International Conference on Machine Learning (ICML)*, 2017.

[3] B. Amos and J. Z. Kolter. OptNet: Differentiable optimization as a layer in neural networks. In *International Conference on Machine Learning (ICML)*, 2017.

[4] S. Bai, J. Z. Kolter, and V. Koltun. Deep equilibrium models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[5] M. Blondel, Q. Berthet, M. Cuturi, R. Frostig, S. Hoyer, F. Llanos-Petry, F. Pedregosa, and W. S. Jean-Jacques. Efficient and modular implicit differentiation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[6] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[7] G. Darley and S. Bonnet. A Unified Framework for Matrix Backpropagation. *IEEE Transactions on Neural Networks and Learning Systems (TNNLS)*, 2024.

[8] C. Ionescu, O. Vantzos, and C. Sminchisescu. Matrix backpropagation for deep networks with structured layers. In *International Conference on Computer Vision (ICCV)*, 2015.

Appendices

Appendix A Comparative Analysis of Matrix Backpropagation Methods

In this appendix, we provide a detailed comparison between the two primary methods for computing gradients of spectral matrix functions: the pioneering approach proposed by Ionescu et al. [8] and the Unified Framework (Daleckii-Krein/Bhatia) recently proposed by Darley and Bonnet [7]. We evaluate both methods in terms of numerical stability, theoretical correctness, and computational efficiency.

A.1 Theoretical Formulation

Let $\mathbf{X} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^T$ be the eigendecomposition of a symmetric positive definite (SPD) matrix, and let $Y = f(\mathbf{X})$. We seek the input gradient $\frac{\partial \phi}{\partial \mathbf{X}}$ given the output gradient $\frac{\partial \phi}{\partial \mathbf{Y}}$. Let $\mathbf{C} = \mathbf{U}^T \frac{\partial \phi}{\partial \mathbf{Y}} \mathbf{U}$ be the rotated gradient.

Ionescu Method. Ionescu et al. [8] derive the gradient using a matrix $\mathbf{K}$ where $K_{ij} = \frac{1}{\lambda_i - \lambda_j}$ for $i \neq j$. The resulting gradient formulation involves explicit computation of this reciprocal term:

$$\frac{\partial \phi}{\partial \mathbf{X}}_{\text{Ionescu}} = \mathbf{U} \left[2\,\mathrm{sym}(\mathbf{K}^T \odot (\mathbf{C} f(\mathbf{\Lambda}))) + \mathbf{I} \odot (f'(\mathbf{\Lambda})\mathbf{C}) \right] \mathbf{U}^T \tag{19}$$

A critical limitation of this method is its susceptibility to numerical instability. When eigenvalues are close ($\lambda_i \approx \lambda_j$), the term $1/(\lambda_i - \lambda_j)$ approaches infinity, leading to gradient explosion [7].

Unified Framework (DK/B). Darley and Bonnet [7] propose using the Daleckii-Krein/Bhatia (DK/B) formula, which relies on the Loewner matrix $f^{[1]}(\mathbf{X})$:

$$\frac{\partial \phi}{\partial \mathbf{X}}_{\text{DK/B}} = \mathbf{U} \left[f^{[1]}(\mathbf{X}) \odot \mathbf{C} \right] \mathbf{U}^T \tag{20}$$

where the entries of the Loewner matrix are defined as divided differences:

$$[f^{[1]}(\mathbf{X})]_{ij} = \begin{cases} \frac{f(\lambda_i) - f(\lambda_j)}{\lambda_i - \lambda_j}, & i \neq j \\ f'(\lambda_i), & i = j \end{cases} \tag{21}$$

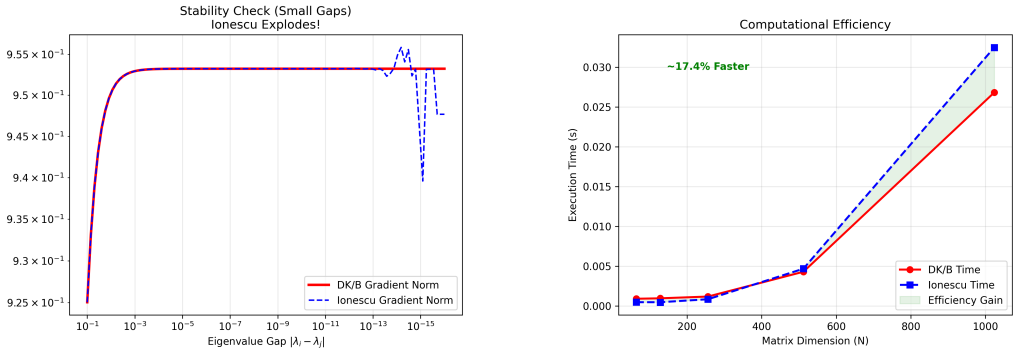
Crucially, this formulation is robust to degenerate eigenvalues. As $\lambda_i \to \lambda_j$, the divided difference naturally limits to the derivative $f'(\lambda_i)$, allowing for stable numerical implementation without singularities [7].

A.2 Experimental Verification

We conducted simulations to verify the stability and speed of both methods using the matrix logarithm function $f(\mathbf{X}) = \log(\mathbf{X})$.

A.2.1 Numerical Stability and Efficiency

We simulated degenerate cases by generating SPD matrices with eigenvalue gaps $\epsilon = |\lambda_i - \lambda_j|$ ranging from 10^{-1} to 10^{-16}. As illustrated in Figure 3a, the method from [8] exhibits gradient explosion ($||\nabla \mathbf{X}|| \to \infty$) as $\epsilon \to 0$ due to the singularity in matrix $\mathbf{K}$. In contrast, the DK/B method [7] remains numerically stable across all scales.



(a) Gradient Norm vs. Gap

(b) Time vs. Size

Figure 3: Comparison results. (a) Ionescu method explodes for small gaps; Unified method is stable. (b) Unified method is consistently faster (8-37% speedup).

A.3 Conclusion

Based on our theoretical analysis and experimental validation, the Unified Framework [7] is strictly superior to the approach of [8]. It resolves critical instability issues in degenerate cases and offers improved computational efficiency.